

10주차

뷰·인덱스·권한(DCL)

경영 리포트 뷰 · 조회 성능 · 권한 최소화

한국기술교육대학교 경영학부

경영데이터베이스및실습 (CCT770) · 2026학년도 2학기

이번 주를 마치면

목표 ①	뷰의 개념·장단점을 알고 경영 리포트 뷰를 만들어 쓴다
목표 ②	뷰의 갱신 가능 조건과 제약을 설명한다
목표 ③	인덱스 동작 원리와 인덱스를 못 타는 질의 패턴을 구분한다
목표 ④	EXPLAIN 실행 계획으로 인덱스 사용 여부를 판단한다
목표 ⑤	GRANT · REVOKE · ROLE 로 계정 권한을 최소한으로 관리한다

다운유통 DB에 리포트 뷰 · 인덱스 · 계정 권한을 직접 걸어 봅니다

오늘 다루는 일곱 가지

1	뷰의 개념 — 경영 리포트 뷰	뷰	5	EXPLAIN 과 인덱스 설계 지침	인덱스
2	뷰의 장단점과 갱신 가능 조건	뷰	6	권한 최소화 — GRANT / REVOKE	DCL
3	인덱스의 개념과 종류	인덱스	7	뷰 + 권한, 롤(ROLE)	DCL
4	인덱스가 사용되지 않는 경우	인덱스			

1. 뷰 (View)

뷰 — 이름을 붙여 저장한 SELECT

가상 테이블

01

하나 이상의 테이블에서
유도된 Virtual Table

정의만 저장

02

실제 데이터가 아니라
SELECT 문의 정의만 저장

조회할 때 계산

03

조회하는 시점에
원본 테이블에서 가져온다

매번 긴 SQL을 다시 짜지 않고, 자주 보는 리포트를 이름 하나로 호출하는 도구

1. 뷰 (View)

경영 리포트 뷰 — 월별 매출 뷰

```
1 CREATE VIEW v_monthly_sales AS
2 SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS 월,
3         SUM(i.qty * i.unit_price)          AS 매출액
4 FROM   orders o
5 JOIN   order_items i ON o.order_id = i.order_id
6 WHERE  o.status <> '취소'
7 GROUP BY DATE_FORMAT(o.order_date, '%Y-%m');
8
9 SELECT * FROM v_monthly_sales ORDER BY 월;
```

1행

- 1 v_ 접두어
테이블과 구분하는 관례

2행

- 2 연-월만 뽑는다
'2026-01' 형태

6행

- 3 취소 제외를 뷰에 고정
누구도 매출을 부풀릴 수 없다

9행

- 4 테이블처럼 조회
실행할 때마다 원본을 다시 읽는다

복잡한 조인 · 집계를 뷰로 캡슐화 — 반복되는 경영 규칙은 뷰에 고정하는 것이 안전하다

1. 뷰 (View)

뷰 수정 · 삭제 · 목록

하려는 일	SQL	알아 둘 점
정의 바꾸기	CREATE OR REPLACE VIEW v_monthly_sales AS SELECT ...;	없으면 새로 만든다
뷰 삭제	DROP VIEW IF EXISTS v_monthly_sales;	원본 데이터는 그대로
뷰 목록	SHOW FULL TABLES WHERE table_type = 'VIEW';	현재 DB 의 뷰
정의 확인	SHOW CREATE VIEW v_monthly_sales;	저장된 SELECT 문

뷰를 지워도 orders · order_items 의 데이터는 하나도 사라지지 않는다

1. 뷰 (View)

VIP 고객 뷰 — 조건을 이름으로

```
1 CREATE VIEW v_vip_customers AS
2 SELECT cust_id, name, city, points
3 FROM customers
4 WHERE grade = 'VIP';
5
6 SELECT * FROM v_vip_customers ORDER BY points DESC;
```

2행

- 1 필요한 컬럼만 노출
email 등은 뷰에 없어 보이지 않는다

4행

- 2 등급 조건을 뷰에
등급이 바뀌면 결과도 따라간다

6행

- 3 포인트 많은 VIP 순
우대 혜택 대상 파악

자주 쓰는 WHERE 조건을 뷰에 담아 두면 「VIP 명단」이 한 단어가 된다

1. 뷰 (View)

뷰의 장단점

장점	단점
보안 — 민감한 컬럼(이메일·포인트 등)을 숨기고 필요한 부분만 노출	독립적인 인덱스를 만들 수 없다
편의성 — 복잡한 조인·집계를 단순한 이름으로 사용	갱신(INSERT/UPDATE/DELETE)에 제약이 있다
논리적 독립성 — 원본 구조가 바뀌어도 뷰 정의만 수정	ALTER로 컬럼 구조를 직접 바꿀 수 없다 (재생성 필요)
질의 재사용으로 유지보수성 향상	뷰가 중첩되면 성능 예측이 어려워진다

뷰는 보안 · 편의성이 장점이고, 인덱스 · 갱신 · 구조 변경에 제약이 있다

1. 뷰 (View)

갱신할 수 없는 뷰 — 행이 1:1로 대응하지 않을 때

- 1 집계 함수(SUM, COUNT, AVG ...) 또는 GROUP BY / HAVING
- 2 DISTINCT, UNION, UNION ALL
- 3 서브쿼리가 SELECT 목록에 있는 경우, 일부 조인 형태
- 4 원본 테이블의 NOT NULL 컬럼이 뷰에 포함되지 않은 경우 (INSERT 불가)

뷰를 통한 변경은 뷰의 행과 원본의 행이 1:1로 대응할 때만 — v_monthly_sales는 집계 뷰라 조회 전용

1. 뷰 (View)

WITH CHECK OPTION — 뷰 조건을 지키게 강제

```
1 CREATE VIEW v_vip_only AS
2 SELECT cust_id, name, grade, city, email, joined_date, points
3 FROM customers
4 WHERE grade = 'VIP'
5 WITH CHECK OPTION;
6
7 INSERT INTO v_vip_only
8 VALUES ('C011', '오하늘', 'SILVER', '서울',
9         'haneul@example.com', '2026-07-01', 0);
10 -- ERROR 1369: CHECK OPTION failed (grade='VIP' 조건 위반)
```

2행

- 1 원본의 모든 컬럼
행이 1:1 대응 → 갱신 가능한 뷰

5행

- 2 조건 밖 행은 거부
VIP 아닌 행은 INSERT/UPDATE 불가

8-9행

- 3 grade 가 SILVER
실제로는 customers 에 들어갈 행

옵션이 없었다면 SILVER 고객이 조용히 저장되고 뷰에서는 보이지도 않는다 — 데이터 무결성 사고

2. 인덱스 (Index)

인덱스 — 책의 색인

정렬된 위치 정보

01

데이터의 위치를 별도로
정렬해 저장한 구조

읽기는 빠르게

02

검색 속도를
크게 높여 준다

쓰기는 느리게

03

저장 공간을 더 쓰고
INSERT·UPDATE·DELETE 때 함께
갱신

주문이 수십만 건 쌓인 뒤 「특정 고객의 주문 이력」 조회가 느려졌다면 가장 먼저 살펴볼 것

2. 인덱스 (Index)

인덱스의 종류

인덱스 종류	설명
클러스터형 인덱스	데이터 자체를 키 순서로 정렬 저장. InnoDB에서 PK가 자동으로 이 역할
보조(비클러스터형) 인덱스	키와 PK 값만 저장. 테이블당 여러 개 생성 가능
복합(결합) 인덱스	여러 컬럼을 묶은 인덱스. 왼쪽 컬럼부터 순서대로 사용됨
UNIQUE 인덱스	중복을 허용하지 않는 인덱스 (UNIQUE 제약을 걸면 자동 생성)
커버링 인덱스	질의에 필요한 모든 컬럼이 인덱스에 포함되어 테이블 접근이 불필요
전문(FULLTEXT) 인덱스	긴 문자열의 단어 검색용

복합 인덱스는 왼쪽 컬럼부터 순서대로 사용된다

2. 인덱스 (Index)

인덱스 생성과 관리

```
1 CREATE INDEX idx_orders_date ON orders(order_date);
2
3 CREATE INDEX idx_orders_cust_date ON orders(cust_id, order_date);
4
5 SHOW INDEX FROM orders;
6 DROP INDEX idx_orders_date ON orders;
```

1

1행

1 단일 컬럼 인덱스

기간별 매출 조회가 잦다면 주문일에

2

3행

2 복합 인덱스 — 순서가 중요

= 조건(cust_id) + 범위(order_date)

3

4

5행

3 구성 확인

Key_name · Column_name · Seq_in_index

6행

4 안 쓰는 인덱스 삭제

쓰기 부담을 줄인다

PK 와 UNIQUE 제약(customers.email)은 이미 인덱스가 있으므로 다시 만들지 않는다 · 이름 관례 idx_테이블_컬럼

2. 인덱스 (Index)

인덱스가 사용되지 않는 다섯 가지

■ 인덱스를 못 타는 조건

■ 이유

■ 바꿔 쓰기

❗ ① 인덱스 컬럼에 함수·연산



행마다 계산해 정렬 순서를 못 쓴다



✓ 컬럼은 그대로, 범위 조건으로

❗ ② LIKE 패턴이 % 로 시작



시작 글자를 몰라 처음부터 끝까지 훑음



✓ '무선%' 처럼 앞을 고정

❗ ③ 부정 조건 (!=, NOT IN, IS NOT NULL)



「아닌 것」은 대부분의 행



✓ 대부분 전체 스캔이 낫다고 판단

❗ ④ 복합 인덱스 선행 컬럼 생략



왼쪽 컬럼부터 순서대로만 쓸 수 있다



✓ 선행 컬럼(cust_id)을 조건에

❗ ⑤ 자료형이 다른 값과 비교



암시적 형 변환으로 인덱스 무효



✓ 'C001' 처럼 자료형에 맞게

2. 인덱스 (Index)

① 함수 적용 · ② 앞쪽 % LIKE

```
1 -- ① 인덱스 컬럼에 함수·연산을 적용
2 SELECT * FROM orders WHERE YEAR(order_date) = 2026;
3 SELECT * FROM orders WHERE order_date >= '2026-01-01'
4                               AND order_date < '2027-01-01';
5
6 -- ② LIKE 패턴이 %로 시작
7 SELECT * FROM products WHERE name LIKE '%노트';
8 SELECT * FROM products WHERE name LIKE '무선%';
```

2행

- 1 (X) 행마다 YEAR() 계산
정렬 순서를 이용 못 함

3-4행

- 2 (O) 범위 조건
컬럼은 그대로 — 결과는 같다

7행

- 3 (X) 시작 글자를 모른다
색인을 처음부터 끝까지 훑는다

8행

- 4 (O) '무선' 으로 시작
그 구간만 찍어 읽는다

인덱스는 '값 그대로' 정렬된 색인 — 값을 가공하거나 앞부분이 불확실하면 색인을 못 쓴다

2. 인덱스 (Index)

③ 부정 조건 · ④ 선행 컬럼 · ⑤ 형 변환

```
1 -- ③ 부정 조건 (!=, NOT IN, IS NOT NULL)
2 SELECT * FROM orders WHERE status != '취소';
3
4 -- ④ 복합 인덱스의 선행 컬럼을 조건에서 생략
5 SELECT * FROM orders WHERE cust_id = 'C001';
6 SELECT * FROM orders WHERE order_date = '2026-01-10';
7
8 -- ⑤ 자료형이 다른 값과 비교 (암시적 형 변환)
9 SELECT * FROM customers WHERE cust_id = 1001;
```

1

2행

- 1 대부분 (X)
「아닌 것」은 대부분의 행

2

5행

- 2 (O) 선행 컬럼 사용
인덱스 (cust_id, order_date)

3

6행

- 3 (X) 선행 컬럼 누락
두 번째 컬럼만 조건에 있음

4

9행

- 4 (X) cust_id 는 CHAR
'C001' 처럼 따옴표로 비교

④ 는 전화번호부에서 이름 없이 생년월일만으로 찾는 것과 같다

2. 인덱스 (Index)

EXPLAIN — 실행 계획 확인

```
1 EXPLAIN SELECT * FROM orders WHERE cust_id = 'C001';  
2 EXPLAIN ANALYZE SELECT * FROM orders WHERE cust_id = 'C001';
```

1행

- ① 실행하지 않고 계획만
type · key · rows 를 본다

2행

- ② 실제 실행 시간까지
진짜로 실행 — UPDATE·DELETE 는 신중히

실습 포인트: 인덱스를 만들기 전 · 후에 EXPLAIN 을 찍어 type · key · rows 가 어떻게 바뀌는지 비교한다

EXPLAIN 결과 읽기

EXPLAIN 컬럼	의미	주목할 값
type	접근 방식	const > eq_ref > ref > range > index > ALL(전체 스캔, 나쁨)
key	실제 사용된 인덱스	NULL이면 인덱스 미사용
rows	읽을 것으로 예상되는 행 수	작을수록 좋음
filtered	조건 통과 비율(%)	낮으면 불필요한 행을 많이 읽음
Extra	부가 정보	Using index(커버링, 좋음), Using filesort·temporary(주의)

type=ALL 과 key=NULL 은 전체 테이블 스캔 — 인덱스를 쓰지 않았다는 뜻

2. 인덱스 (Index)

인덱스 설계 지침

- ☐ WHERE · JOIN · ORDER BY 에 자주 쓰이는 컬럼에 만든다
- ☐ 카디널리티(값의 다양성)가 높은 컬럼이 효과적이다
customers.grade 처럼 VIP/GOLD/SILVER 3종류뿐이면 효과가 거의 없다
- ☐ 복합 인덱스는 선택도가 높은 컬럼(= 조건)을 앞에, 범위 조건 컬럼을 뒤에
- ☐ 인덱스가 많을수록 쓰기가 느려지므로 사용되지 않는 인덱스는 제거한다
- ☐ 테이블이 매우 작으면 전체 스캔이 오히려 빠를 수 있다
실습 DB의 주문 15건 규모에서는 차이를 체감하기 어렵다

3. 사용자 권한 관리 (DCL)

권한 최소화 원칙

GRANT

계정에 권한을
부여한다

01

REVOKE

계정에서 권한을
회수한다

02

필요한 최소한만

업무에 필요한 만큼만
준다

03

재고 확인만 하는 아르바이트 계정에 DELETE 권한이 있으면, 실수 한 번에 주문 데이터가 통째로 사라질 수 있다

3. 사용자 권한 관리 (DCL)

접근 통제의 세 가지 모델

모델	결정 주체	특징
DAC (임의적 접근통제)	데이터 소유자	GRANT/REVOKE로 권한을 자유롭게 부여. 대부분의 DBMS
MAC (강제적 접근통제)	시스템 관리자	데이터와 사용자에게 보안 등급을 부여. 군사·정부 시스템
RBAC (역할 기반 접근통제)	역할(Role)	역할에 권한을 모으고 사용자에게 역할을 부여. 관리가 쉬움

MySQL 의 GRANT/REVOKE 는 DAC, ROLE 은 RBAC 에 해당한다

3. 사용자 권한 관리 (DCL)

GRANT — 누구에게 · 어디에 · 무엇을

```
1 CREATE USER 'parttime'@'localhost' IDENTIFIED BY 'pt_2026!';
2
3 GRANT SELECT ON daon.products TO 'parttime'@'localhost';
4 GRANT SELECT, INSERT, UPDATE ON daon.* TO 'manager'@'localhost';
5 GRANT ALL PRIVILEGES ON daon.* TO 'admin'@'localhost' WITH GRANT OPTION;
```

1

1행

- 1 '사용자'@'호스트'
한 쌍이 계정 하나

2

3

4

3행

- 2 테이블 하나 · SELECT 만
아르바이트 계정

4행

- 3 daon 전체 · DELETE 없음
매니저 계정

5행

- 4 WITH GRANT OPTION
받은 권한을 남에게 다시 부여

권한 최소화: 업무에 필요한 만큼만 — 실수 · 유출의 피해 범위를 계정 단위로 묶는다

3. 사용자 권한 관리 (DCL)

컬럼 단위 권한 · 확인 · 회수

```
1 GRANT SELECT (cust_id, name, grade) ON daon.customers
2   TO 'parttime'@'localhost';
3
4 SHOW GRANTS FOR 'parttime'@'localhost';
5 REVOKE INSERT, UPDATE ON daon.* FROM 'manager'@'localhost';
6 REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'admin'@'localhost';
7
8 FLUSH PRIVILEGES;
```

1

1-2행

1

컬럼 단위 권한

email · points 를 SELECT 하면 거부

2

5행

2

준 것과 같은 범위로 회수

daon.* — SELECT 는 남는다

3

6행

3

한 번에 회수

모든 권한과 재부여 권한

4

8행

4

보통 필요 없다

GRANT/REVOKE 문은 즉시 반영

SHOW GRANTS 로 계정의 권한 목록 확인 · FLUSH PRIVILEGES 는 권한 테이블을 직접 고쳤을 때 다시 읽게 하는 명령

3. 사용자 권한 관리 (DCL)

뷰 + 권한 — 민감한 고객 정보 보호

```
1 CREATE VIEW v_customer_basic AS
2 SELECT cust_id, name, grade, city
3 FROM customers;
4
5 GRANT SELECT ON daon.v_customer_basic TO 'parttime'@'localhost';
6
7 SELECT * FROM v_customer_basic;
8 SELECT * FROM customers;
```

1

2행

- 1 업무에 필요한 4개 컬럼
email · points 는 뷰 정의에 없다

2

5행

- 2 원본 대신 뷰에만
뷰도 권한 대상이 된다

3

4

7행

- 3 (O) 뷰 조회
권한 검사는 뷰 기준

8행

- 4 (X) 원본 조회
ERROR 1142: SELECT command denied

계정 입장에서는 처음부터 그 컬럼이 존재하지 않는 것과 같다 — 뷰의 보안 장점과 GRANT 가 만나는 지점

3. 사용자 권한 관리 (DCL)

롤(ROLE) — 권한을 역할로 묶기 (MySQL 8.0)

```
1 CREATE ROLE 'role_read', 'role_write';
2
3 GRANT SELECT ON daon.* TO 'role_read';
4 GRANT INSERT, UPDATE, DELETE ON daon.* TO 'role_write';
5
6 GRANT 'role_read' TO 'parttime'@'localhost';
7 GRANT 'role_read', 'role_write' TO 'manager'@'localhost';
8
9 SET DEFAULT ROLE ALL TO 'manager'@'localhost';
10
11 REVOKE 'role_write' FROM 'manager'@'localhost';
12 DROP ROLE 'role_write';
```

1

1행

- 1 역할 두 개
로그인은 못 한다

2

6-7행

- 2 사람에게는 역할만
아르바이트 = 읽기 · 매니저 = 읽기+쓰기

3

9행

- 3 로그인 때 자동 활성화
없으면 SET ROLE 을 직접

4

12행

- 4 역할 자체 삭제
가진 모든 계정에서 권한이 사라진다

직원이 바뀌어도 역할만 붙이고 떼면 된다 — RBAC 구현

3. 사용자 권한 관리 (DCL)

권한 범위 표기

권한 범위	표기	예시
전역	*.*	모든 데이터베이스의 모든 객체
데이터베이스	daon.*	daon 안의 모든 테이블
테이블	daon.customers	특정 테이블
컬럼	SELECT (name, grade)	특정 컬럼만

범위는 좁을수록 안전하다 — 테이블 · 컬럼 단위까지 나눌 수 있다

확인 퀴즈

customers.joined_date에 인덱스가 있을 때, 인덱스를 사용할 수 없게 만드는 조건은?

① WHERE name LIKE '김%'

② WHERE joined_date >= '2025-01-01'

③ WHERE YEAR(joined_date) = 2025

④ WHERE grade = 'VIP'

정답 및 해설

정답

③ WHERE YEAR(joined_date) = 2025

해설

인덱스 컬럼에 함수를 적용하면 인덱스를 사용할 수 없습니다. YEAR(joined_date) = 2025 대신 joined_date BETWEEN '2025-01-01' AND '2025-12-31' 같은 범위 조건으로 바꿔 작성해야 합니다.

핵심 요약

핵심 ①	뷰는 저장 공간이 없는 가상 테이블, 독립 인덱스를 못 만든다
핵심 ②	집계 · GROUP BY · DISTINCT · UNION 이 든 뷰는 갱신 불가
핵심 ③	인덱스는 읽기를 빠르게, 쓰기를 느리게 — 카디널리티 높은 컬럼에
핵심 ④	함수 적용 · 앞쪽 % LIKE · 선행 컬럼 누락이면 인덱스를 못 쓴다
핵심 ⑤	EXPLAIN 의 type=ALL 과 key=NULL 은 전체 테이블 스캔
핵심 ⑥	권한은 필요한 최소한만 — 민감 컬럼은 뷰 + 뷰에만 GRANT

조회 계정에는 **SELECT** 만, 반복되는 경영 규칙은 뷰에 고정

다음 주차 예고

11주차 · 설계와 정규화

01

설계 절차

요구분석 → 개념
→ 논리 → 물리

02

E-R 모델

ERD 작성법과
1:N · M:N 변환

03

이상 현상

함수적 종속으로
원인 진단

04

정규화

1NF → 2NF → 3NF
다운유통 스키마

10주차 마무리

학습 마무리

오늘 만든 뷰 · 인덱스 · 계정을 다운유통 실습 DB에서 확인해 봅니다

✓ 경영 리포트 뷰와 WITH CHECK OPTION

✓ 인덱스를 못 타는 다섯 패턴

✓ EXPLAIN 으로 실행 계획 읽기

✓ GRANT · REVOKE · ROLE

이어지는 활동

10주차 실습 문제 · 퀴즈 8